

26. Browser Events: Introduction

Event = ek signal ki kuch ho gaya. Saare DOM nodes events generate karte hain.

Kuch useful events

- **Mouse:** `click`, `contextmenu` (right click), `mouseover/mouseout`, `mousedown/mouseup`, `mousemove`
- **Keyboard:** `keydown`, `keyup`
- **Form:** `submit`, `focus`
- **Document:** `DOMContentLoaded` (DOM pura ban gaya)
- **CSS:** `transitionend`

Handler lagane ke 3 tarike

1. HTML attribute:

html

```
<input type="button" onclick="alert('Click!')" value="Click me">
```

Code bada ho to function banake call karo (`onclick="countRabbits()"`). Attribute me double quote ke andar single quote use karo.

2. DOM property:

js

```
elem.onclick = function() { alert('Thank you'); };
```

- Sirf **ek** handler (`onclick`) lag sakta hai, naya purane ko **overwrite** kar deta hai.
- Hatane ke liye `elem.onclick = null`.
- Case-sensitive: `onclick`, `ONCLICK` nahi.

3. `addEventListener` (sabse flexible):

js

```
element.addEventListener(event, handler, [options]);  
element.removeEventListener(event, handler, [options]);
```

- **Kai handlers** ek hi event par lag sakte hain.
- Options: `once` (ek baar chal ke hat jaaye), `capture`, `passive`.

- **Remove karne ke liye wahi function object dena padta hai** (naya arrow function alag object hota hai, isliye kaam nahi karega). Function ko variable me rakho.
- Kuch events (jaise `DOMContentLoaded`) sirf `addEventListener` se hi chalte hain.

`this` handler ke andar

`this` wo **element** hota hai jisme handler laga hai.

html

```
<button onclick="alert(this.innerHTML)">Click me</button>
```

Common galtiyen

js

```
button.onclick = sayThanks; // sahi
button.onclick = sayThanks(); // galat (function chal jaata hai, result assign)
```

- HTML attribute me bracket **chahiye** (`onclick="sayThanks()"`), kyunki wo function ki body ban jaati hai.
- `setAttribute('onclick', function...)` **mat** karo (function string ban jaata hai).

Event object

Browser event ke bare me details ka object bana kar handler ko pehle argument me deta hai:

js

```
elem.onclick = function(event) {
  console.log(event.type, event.currentTarget, event.clientX, event.clientY);
};
```

- `event.type`: event ka type
- `event.currentTarget`: handler wala element (`this` jaisa, par arrow function me bhi kaam karta hai)
- `event.clientX / clientY`: window-relative cursor coordinates
- HTML handlers me bhi `event` available hota hai.

Object handlers: `handleEvent`

`addEventListener` me function ki jagah **object** bhi de sakte ho. Event aane par uska `handleEvent(event)` method chalta hai:

js

```
class Menu {
  handleEvent(event) {
    let method = 'on' + event.type[0].toUpperCase() + event.type.slice(1);
    this[method](event);
  }
  onMousedown() { ... }
  onMouseup() { ... }
}
let menu = new Menu();
elem.addEventListener('mousedown', menu);
elem.addEventListener('mouseup', menu);
```

Summary

Tarika	Limit
HTML attribute	Zyada code nahi likh sakte
DOM property	Sirf ek handler
<code>addEventListener</code>	Sabse flexible, thoda lamba

Tasks ke jawab (short)

- `button.addEventListener("click", () => alert("1"));`
`button.removeEventListener("click", () => alert("1")); button.onclick = () => alert(2);` -> **1 aur 2 dono** challenge (removeEventListener ko naya function mila).
- Button khud ko chhupaye: `<input type="button" onclick="this.hidden=true" value="Click to hide">`
- Menu open/close: CSS me `.open` class toggle karo, JS sirf class badalta hai.